University of Asia Pacific

Department of Computer Science & Engineering

Course Name: Artificial Intelligence & Expert Systems Lab

Course Code: CSE 404

Report No.: 3

Project Title: Adversarial Search: Tic Tac Toe with Minimax & Alpha-Beta pruning

Date of Submission: 18-09-2025

Submitted by:

Name: Nafis Fuad Barshan

Reg. No.:21201166

Name: Tanvir Rahan Rifat

Reg. No.:22101065

Name: MD. Adnan Asif

Reg. No.:22101081

Submitted To:

Nahida Marzan

Lecturer

Department of Computer Science & Engineering

University of Asia Pacific

Problem Title: Adversarial Search: Tic Tac Toe with Minimax & Alpha-Beta pruning

Problem Description

The goal of this project is to implement the classic Tic Tac Toe game as an adversarial search problem. A Tic Tac Toe board has 3 rows and 3 columns, and two players take turns placing their marks (X or O). The objective is to align three of the same marks horizontally, vertically, or diagonally.

We extend the basic game by implementing an AI player that uses the Minimax algorithm with Alpha-Beta pruning to always play optimally.

The project supports:

- Human vs Computer (interactive mode).
- Computer vs Computer (simulation mode).
- Printing the board after every move and declaring the result.

Tools and Technologies

To implement and execute the Tic Tac Toe game with adversarial search, the following tools and technologies were used:

1. **Programming Language – Python**
    - Python was chosen as the core programming language because of its simplicity, readability, and extensive support for algorithm implementation.
    - Its built-in libraries and straightforward syntax made it easy to implement the minimax algorithm with alpha-beta pruning efficiently.
2. **Google Colaboratory (Colab)**
    - Google Colab was used as the development and execution environment.
    - It provides a cloud-based Jupyter Notebook interface, allowing the program to be written, tested, and executed without the need for local setup.
    - Features like code cells, console-based input/output, and free GPU/CPU resources made it suitable for experimentation and demonstration.
3. **Python Standard Libraries**
    - **math**: Used for mathematical constants and functions, particularly math.inf to represent positive and negative infinity in the minimax algorithm.
    - **random**: Used for handling randomization if needed (e.g., breaking ties between equally optimal moves).
4. **Console-based Interface**
    - A simple text-based console interface was used instead of a graphical interface, keeping the focus on the core logic of adversarial search.
    - This interface allows players to enter moves (row and column positions) and displays the board state after each move.

Tic Tac Toe Overview

- A 3x3 board game played between two players.
- Player **X** and Player **O** place their marks alternately.
- A player wins if they manage to place three consecutive marks in a row, column, or diagonal.
- If the board is full and no player wins, the result is a draw.

Minimax Algorithm with Alpha-Beta Pruning

Minimax Concept

- A recursive decision-making algorithm used for two-player adversarial games.
- The Maximizer (AI) tries to maximize the score.
- The Minimizer (Human) tries to minimize the score.
- The algorithm explores all possible moves and counter-moves until a terminal state (win, lose, draw) is reached.
- Scores:
  - +10 → AI (O) wins.
  - –10 → Human (X) wins.
  - 0 → Draw.

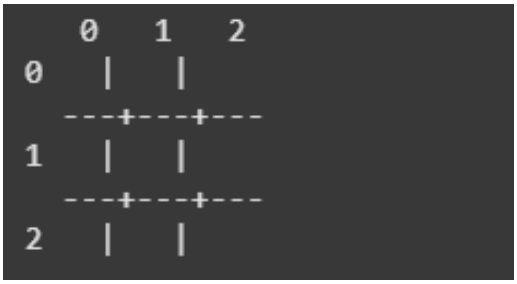
Alpha-Beta Pruning

- Optimization of Minimax.
- alpha → best value Maximizer can guarantee.
- beta → best value Minimizer can guarantee.
- If a branch cannot produce a better result than already found, it is pruned (ignored).
- This significantly reduces the number of nodes evaluated.

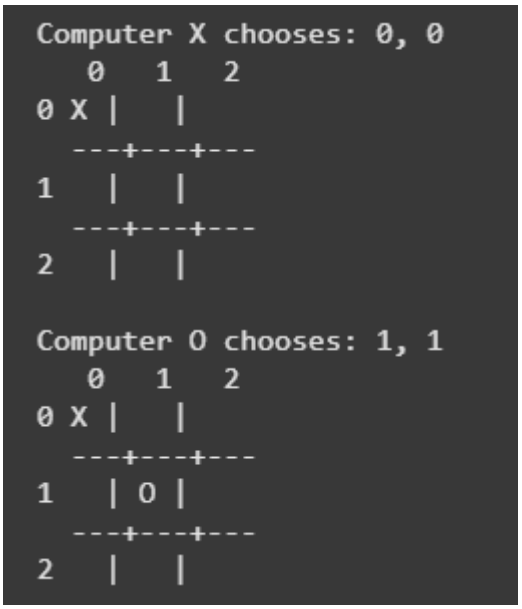
Pseudocode

```
function play_game(mode, depth_x, depth_o):
   initialize empty 3x3 board
   set current_player = "X"

   while True:
      if current_player == "X":
         if mode == "human-vs-computer":
            get move input from human (row, col)
         else:
            move = find_best_move(board, "X", depth_x)
      else:
         move = find_best_move(board, "O", depth_o)

      if chosen cell is empty:
         place current_player mark on board
      else:
         print "Invalid move, retry"
         continue
      print current state of board
      score = evaluate(board)
      if score == +10:
         print "O wins"
         break
      else if score == -10:
         print "X wins"
         break
      else if no moves left:
         print "Draw"
         break
      switch current_player (X ↔ O)
```

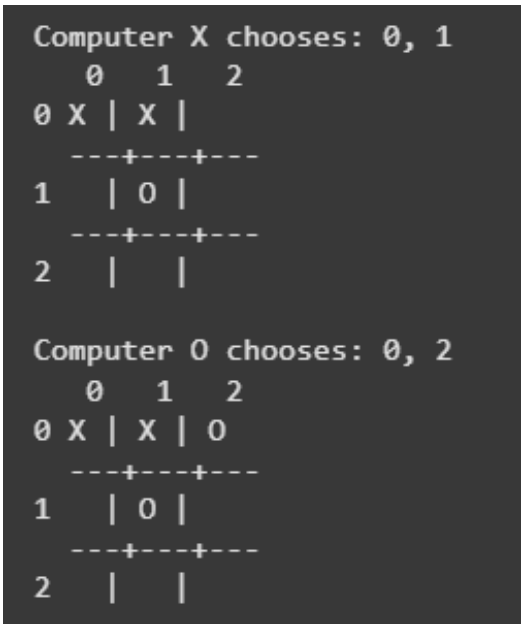
Sample Input/Output (Computer vs Computer)



```
   0   1   2
0    |   |
  ---+---+---
1    |   |
  ---+---+---
2    |   |
```



```
Computer X chooses: 0, 0
   0   1   2
0 X |   |
  ---+---+---
1    |   |
  ---+---+---
2    |   |

Computer O chooses: 1, 1
   0   1   2
0 X |   |
  ---+---+---
1    | O |
  ---+---+---
2    |   |
```



```
Computer X chooses: 0, 1
   0   1   2
0 X | X |
  ---+---+---
1    | O |
  ---+---+---
2    |   |

Computer O chooses: 0, 2
   0   1   2
0 X | X | O
  ---+---+---
1    | O |
  ---+---+---
2    |   |
```

```
Computer X chooses: 2, 0
   0   1   2
0 X | X | O
  ---+---+---
1   | O |
  ---+---+---
2 X |   |

Computer O chooses: 1, 0
   0   1   2
0 X | X | O
  ---+---+---
1 O | O |
  ---+---+---
2 X |   |
```

```
Computer X chooses: 1, 2
   0   1   2
0 X | X | O
  ---+---+---
1 O | O | X
  ---+---+---
2 X |   |

Computer O chooses: 2, 1
   0   1   2
0 X | X | O
  ---+---+---
1 O | O | X
  ---+---+---
2 X | O |
```

```
Computer X chooses: 2, 2
   0   1   2
0 X | X | O
  ---+---+---
1 O | O | X
  ---+---+---
2 X | O | X

It's a draw!
```

Sample Input/Output (Human vs Computer)

```
   0   1   2
0  |   |
  ---+---+---
1  |   |
  ---+---+---
2  |   |
```

```
Enter your move (row col): 0 0
   0   1   2
0 X |   |
  ---+---+---
1   |   |
  ---+---+---
2   |   |

Computer O chooses: 1, 1
   0   1   2
0 X |   |
  ---+---+---
1   | O |
  ---+---+---
2   |   |
```

```
Enter your move (row col): 2 2
   0   1   2
0 X |   |
  ---+---+---
1   | O |
  ---+---+---
2   |   | X

Computer O chooses: 0, 1
   0   1   2
0 X | O |
  ---+---+---
1   | O |
  ---+---+---
2   |   | X
```

```
Enter your move (row col): 1 2
   0   1   2
0 X | O |
  ---+---+---
1   | O | X
  ---+---+---
2   |   | X

Computer O chooses: 0, 2
   0   1   2
0 X | O | O
  ---+---+---
1   | O | X
  ---+---+---
2   |   | X
```

```
 Enter your move (row col): 2 1
    0   1   2
 0 X | O | O
   ---+---+---
 1   | O | X
   ---+---+---
 2   | X | X

 Computer O chooses: 2, 0
    0   1   2
 0 X | O | O
   ---+---+---
 1   | O | X
   ---+---+---
 2 O | X | X

 O wins!
```

Key Challenges Faced

1. **Implementing the Minimax Algorithm Efficiently**
    - Designing a recursive minimax function that correctly evaluates game states was challenging, especially ensuring it terminates properly at win, loss, or draw conditions.
2. **Integrating Alpha-Beta Pruning**
    - Adding alpha-beta pruning required careful handling of the alpha and beta values to cut off unnecessary branches without affecting the correctness of the result.
3. **Balancing Search Depth vs. Performance**
    - Since minimax explores many possible game states, higher search depths made the program slower. Choosing appropriate depths for human vs computer and computer vs computer modes was essential.
4. **Handling Human Input Robustly**
    - Ensuring that invalid or duplicate moves are rejected without crashing the program required additional checks in the main game loop.
5. **Ensuring Fair AI Behavior**
    - Preventing bias toward a particular player and confirming that the AI always makes optimal moves was a key testing challenge.

Conclusion

This project successfully demonstrates how adversarial search techniques can be applied to a classic game like Tic Tac Toe. By implementing the minimax algorithm with alpha-beta pruning, the AI is able to make optimal moves while reducing the computational overhead compared to plain minimax.

The program provides both Human vs Computer and Computer vs Computer modes, showcasing how search depth affects decision-making speed and game outcomes. Through this assignment, we gained practical experience in problem modeling, recursive algorithm design, and the importance of optimization techniques in AI.

Overall, the project reinforces the role of adversarial search in game-playing AI and builds a strong foundation for understanding more complex games and search problems in Artificial Intelligence.